Reviewer Pack — Newton Polytope Vertex Count Bounds SOS Degree for Max-CUT A…

Entry 03c7a9ecad95 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 18:02:30 UTC

Newton Polytope Vertex Count Bounds SOS Degree for Max-CUT Approximation

Entry ID: `03c7a9ecad95` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 18:02:30 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry (Newton Polytopes) **Field B** (complexity object): SOS Degree for Max-CUT Approximation

Statement:

> For any graph G with m edges, the SOS degree d required to approximate Max-CUT with a ratio better than 0.878 must satisfy d ≥ log(m), where the SOS degree is the minimal degree of a sum-of-squares relaxation achieving the approximation ratio.

Rationale (proposer's reasoning):

> The Newton polytope's vertex count reflects the polynomial's structural complexity. A higher vertex count implies a more intricate polynomial, necessitating a higher SOS degree to capture its properties, thereby linking combinatorial graph structure to algebraic complexity.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fbbacb261e47e9f0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `INCONCLUSIVE` against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Newton polytope SOS degree Max-CUT approximation` - `lower bound SOS degree Newton polytope approximation ratio` - `log m SOS degree Max-CUT approximation ratio`

Top relevant hits considered: - [http://arxiv.org/abs/1512.01594v2] Pruning Algorithms for Pretropisms of Newton Polytopes - [http://arxiv.org/abs/1604.01183v2] Approximate Polytope Membership Queries - [http://arxiv.org/abs/2107.00574v1] Trigonometric approximation of the Max-Cut Polytope is star-like - [http://arxiv.org/abs/2409.14294v2] A lower bound theorem for d-polytopes with $2d + 2$ vertices - [http://arxiv.org/abs/quant-ph/0305179v3] Polynomial Degree and Lower Bounds in Quantum Complexity: Collision and Element Distinctness with Small Range - [http://arxiv.org/abs/2308.03822v1] Search for Eccentric Black Hole Coalescences during the Third Observing Run of LIGO and Virgo - [http://arxiv.org/abs/2509.08054v1] GW250114: testing Hawking's area law and the Kerr nature of black holes - [http://arxiv.org/abs/2111.13106v2] Searches for Gravitational Waves from Known Pulsars at Two Harmonics in the Second and Third LIGO-Virgo Observing Runs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n, m):
        edges = set()
        while len(edges) < m:
            u, v = random.sample(range(n), 2)
            if (u, v) not in edges and (v, u) not in edges:
                edges.add((u, v))
        return list(edges)

    def max_cut_polynomial(G):
        n = len(G)
        terms = []
        for subset in range(1 << n):
            term = 0
            for i in range(n):
                if (subset >> i) & 1:
                    term += (-1)**sum((i, j) in G for j in range(i+1, n))
            terms.append(term)
        return sum(terms)
```

```
    def newton_polytope_vertex_count(poly):
        # Placeholder implementation; actual computation depends on the polynomial structure
        return len(poly)

    def sos_degree(poly):
        # Placeholder implementation; actual computation depends on the SOS relaxation
        return 0

    n = random.randint(5, 40)
    m = random.randint(n, n*(n-1)//2)
    G = generate_random_graph(n, m)
    poly = max_cut_polynomial(G)
    vertex_count = newton_polytope_vertex_count(poly)
    d = sos_degree(poly)

    metric_name = "SOS Degree"
    metric_value = d
    instances_tested = 1
    conjecture_holds = d >= math.log(m)
    counterexample = "" if conjecture_holds else f"Graph with {n} nodes and {m} edges; SOS degree

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(3, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(resu
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with {results[0]['instances_tested']} noc
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test timed out before producing results; insufficient data to evaluate support fraction or counterexamples | next: Increase timeout duration and re-run test with extended computation resources

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112461
2	propose	ollama_remote	qwen3:8b	0	0	28933
3	propose	ollama_remote	qwen3:8b	0	0	87532
4	preregistration	ollama_remote	qwen3:8b	0	0	24190
5	novelty	ollama_remote	qwen3:8b	0	0	19820
6	novelty	ollama_remote	qwen3:8b	0	0	24781
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11513
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8195
9	judge	ollama_remote	qwen3:8b	0	0	19769

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 337193 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/03c7a9ecad95.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/03c7a9ecad95.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/03c7a9ecad95.tar.gz` (if generated)